

Tugas TLS Pemrograman

Nama : Farhan Kurniawan

Kelompok : Mariposite

Problem 1 :

The Last Astronaut

1. Penjelasan Logika

Problem The Last Astronaut dapat diselesaikan dengan memodelkan sekumpulan astronot sebagai sebuah struktur data antrean (queue) atau array dinamis berbentuk lingkaran (circular). Di awal, kita memasukkan nomor-nomor astronot dari 1 hingga N ke dalam struktur penyimpanan. Proses eliminasi dilakukan secara berulang: kita menghitung maju sebanyak K langkah dari posisi saat ini. Astronot yang jatuh pada hitungan ke-K akan dikeluarkan dari lingkaran dan dicatat ke dalam daftar urutan eliminasi.

Setelah seorang astronot tereliminasi, aturan khusus diberlakukan pada nilai K: jika nomor astronot yang keluar bernilai genap, maka K bertambah 2; sebaliknya jika ganjil, K berkurang 1. Kita juga perlu memastikan bahwa nilai K tidak pernah kurang dari 2 dengan menerapkan kondisi batas minimal. Proses ini terus berlanjut secara berulang hingga hanya tersisa satu astronot di dalam lingkaran, yang kemudian ditetapkan sebagai astronot terakhir yang bertahan.

2. Pseudocode

Algoritma TheLastAstronaut

Kamus:

N, K: integer

astronot: array of integer

tereliminasi: array of integer

index_sekarang: integer

no_tereliminasi: integer

Algoritma:

Input(N, K)

// Inisialisasi daftar astronot dari 1 sampai N

For i <- 1 to N do

 astronot[i] <- i

Endfor

```
index_sekarang <- 0 // Indeks mulai dari 0 (elemen pertama)
```

```
While panjang(astronot) > 1 do
```

```
    // Menghitung posisi astronot yang akan dieleminasi berdasarkan K
```

```
    index_sekarang <- (index_sekarang + K - 1) mod panjang(astronot)
```

```
    // Ambil nomor astronot yang dieleminasi
```

```
    no_tereliminasi <- astronot[index_sekarang]
```

```
    Simpan no_tereliminasi ke dalam daftar tereliminasi
```

```
    // Hapus astronot dari daftar
```

```
    Hapus elemen di indeks_sekarang dari astronot
```

```
    // Perbarui nilai K berdasarkan nomor yang tereliminasi
```

```
    If no_tereliminasi mod 2 == 0 Then
```

```
        K <- K + 2
```

```
    Else
```

```
        K <- K - 1
```

```
    Endif
```

```
    // Batas minimal K adalah 2
```

```
    If K < 2 Then
```

```
        K <- 2
```

```
    Endif
```

```
Endwhile
```

```
Output("Urutan eliminasi:", tereliminasi)
```

```
Output("Astronot terakhir:", astronot[0])
```

```
EndAlgoritma
```

Problem 2:

Alien In The Middle

1. Penjelasan Logika

Problem Alien-In-The-Middle berfokus pada enkripsi pesan teks berdasarkan pergeseran nilai numerik alfabet dari karakter sebelumnya. Karakter pertama dari pesan asli dibiarkan tetap dan tidak mengalami perubahan. Untuk setiap karakter berikutnya pada indeks ke- i , nilai pergeseran ditentukan oleh nilai alfabet dari karakter tepat sebelumnya (indeks ke- $i-1$).

Sistem penomoran alfabet yang digunakan adalah A=1, B=2, C=3, \dots, Z=26. Nilai ASCII dari karakter huruf kapital dikonversi terlebih dahulu ke nilai posisi alfabetnya (misalnya dengan mengurangi nilai ASCII huruf 'A' lalu ditambah 1). Karakter baru diperoleh dengan menambahkan nilai huruf sebelumnya ke huruf saat ini. Jika hasil penjumlahan melebihi 26 (melewati huruf 'Z'), maka perhitungannya dibungkus kembali menggunakan operasi modulo 26 agar kembali berputar dari huruf 'A'. Proses ini diulang hingga seluruh karakter dalam string pesan selesai diterjemahkan.

2. Pseudocode

Algoritma AlienInTheMiddle

Kamus:

pesan: string

hasil_sandi: string

n: integer, i: integer

val_sebelumnya: integer, val_sekarang: integer, val_baru: integer

Algoritma:

Input(pesan)

n <- panjang(pesan)

If n == 0 Then

Exit

Endif

// Karakter pertama tidak berubah

hasil_sandi[0] <- pesan[0]

val_sebelumnya <- (kode_ascii(pesan[0]) - kode_ascii('A') + 1)

```
For i <- 1 to n - 1 do
    val_sekarang <- (kode_ascii(pesan[i]) - kode_ascii('A') + 1)

    // Geser sebanyak nilai huruf sebelumnya
    val_baru <- val_sekarang + val_sebelumnya

    // Jika melewati Z (26), putar kembali dari A
    While val_baru > 26 do
        val_baru <- val_baru - 26
    Endwhile

    // Konversi kembali ke karakter huruf kapital
    hasil_sandi[i] <- karakter_dari_ascii(val_baru + kode_ascii('A') - 1)

    // Perbarui nilai sebelumnya untuk iterasi berikutnya
    val_sebelumnya <- val_sekarang
Endfor

Output("Pesan sandi:", hasil_sandi)
EndAlgoritma
```